

LangChain v1 源码架构分析

langchain_v1 核心模块设计与流程说明

版本 : v1.2.13 (langchain_v1)

基于 Agent + Middleware 架构的下一代 LangChain 框架

目录

- 一、项目概览与整体架构
- 二、模块结构说明
- 三、核心流程图：Agent 主循环
- 四、核心流程图：create_agent 初始化
- 五、核心流程图：结构化输出处理
- 六、核心时序图：中间件组合模式
- 七、核心类设计说明 — AgentMiddleware
- 八、核心类设计说明 — create_agent 工厂函数
- 九、核心类设计说明 — ModelRequest / ModelResponse
- 十、核心类设计说明 — 结构化输出策略
- 十一、核心类设计说明 — init_chat_model 工厂
- 十二、核心类设计说明 — init_embeddings 工厂
- 十三、内置中间件一览
- 十四、设计哲学与总结

一、项目概览与整体架构

LangChain v1 (langchain_v1) 是 LangChain 框架的下一代重构版本，采用了全新的 Agent + Middleware 架构设计。与传统 LangChain (langchain-classic) 相比，v1 版本做了极大的精简，移除了 Chains、Retrievers、Memory、VectorStores 等模块，转而以中间件模式实现所有横切关注点 (cross-cutting concerns)。

langchain_v1 构建在 LangGraph (状态图引擎) 和 langchain-core (核心抽象层) 之上，通过 StateGraph 构建 Agent 的执行图，将'模型调用-工具执行'循环编排为可编译的状态机。中间件可以在 Agent 生命周期的各个阶段插入自定义逻辑，实现重试、限流、PII 脱敏、摘要、人工审核等功能。

核心设计原则

- 极简 API：对外仅暴露 create_agent() 一个核心入口
- 中间件驱动：所有扩展逻辑通过 AgentMiddleware 子类实现
- 基于 LangGraph：Agent 执行流是一个可编译的 StateGraph
- 类型安全：大量使用泛型、TypedDict、Protocol 确保类型安全
- 同步/异步双模式：所有钩子和处理器均支持同步和异步版本

二、模块结构说明

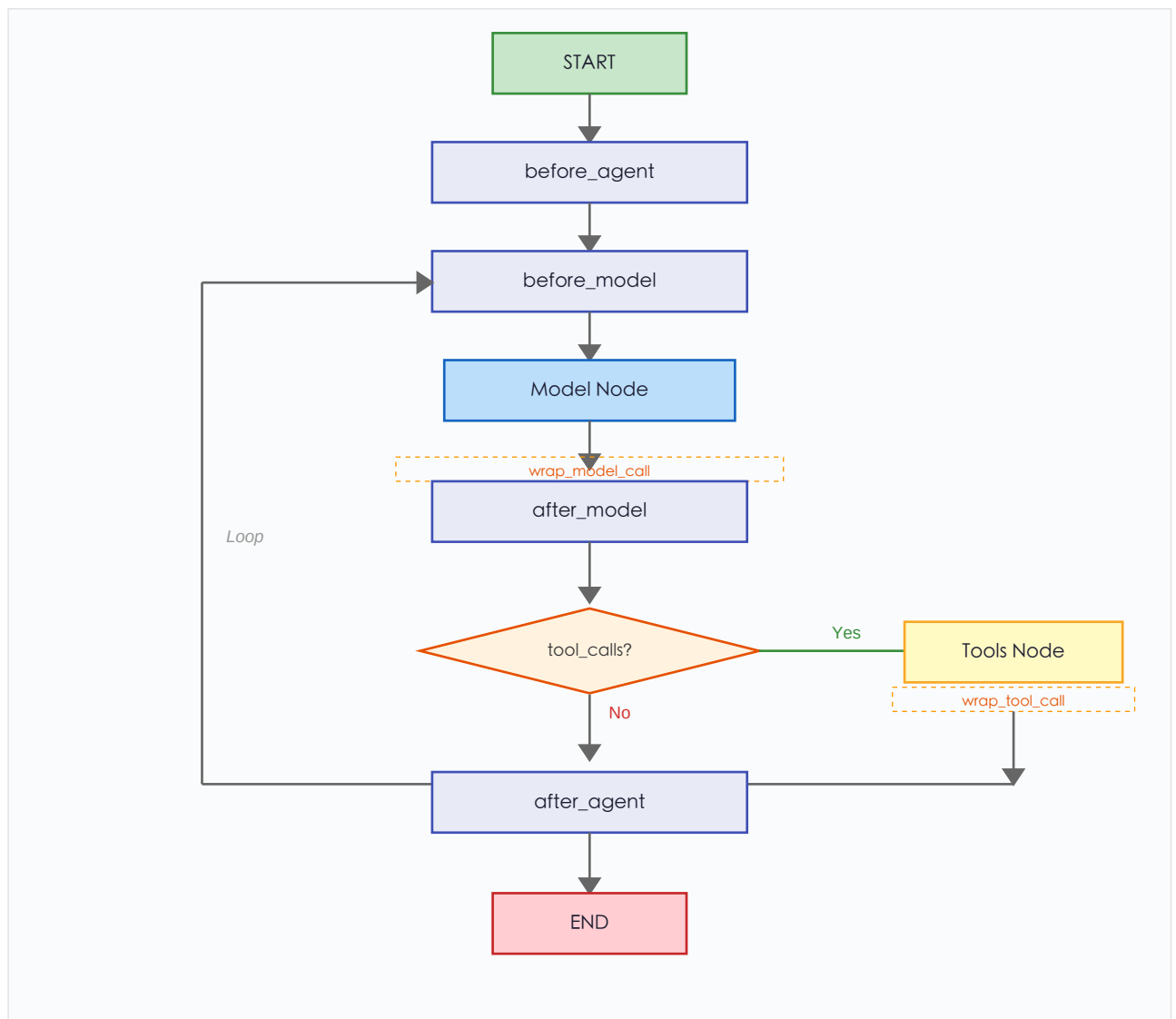
langchain_v1 的源码包非常精简，仅包含 6 个子模块，约 30 个源码文件：

模块	文件数	核心职责
agents/	约 20 个	Agent 工厂、中间件类型系统、17 种内置中间件
chat_models/	2 个	聊天模型工厂 init_chat_model，支持 25+ 提供商
embeddings/	2 个	嵌入模型工厂 init_embeddings，支持 10+ 提供商
tools/	2 个	工具相关类型的重新导出（BaseTool、ToolRuntime 等）
messages/	1 个	消息类型的统一重新导出（AIMessage、ToolMessage 等）
rate_limiters/	1 个	速率限制器的重新导出

其中 agents/ 是最核心、最庞大的模块。factory.py（约 1860 行）是整个框架的核心，middleware/types.py（约 2050 行）定义了完整的中间件类型系统。

三、核心流程图：Agent 主循环

Agent 的核心执行流程是一个'模型调用-工具执行'的循环。当模型返回 `tool_calls` 时，Agent 会执行对应的工具，然后将结果反馈给模型，直到模型不再调用工具为止。中间件可以在循环的各个阶段插入自定义逻辑。



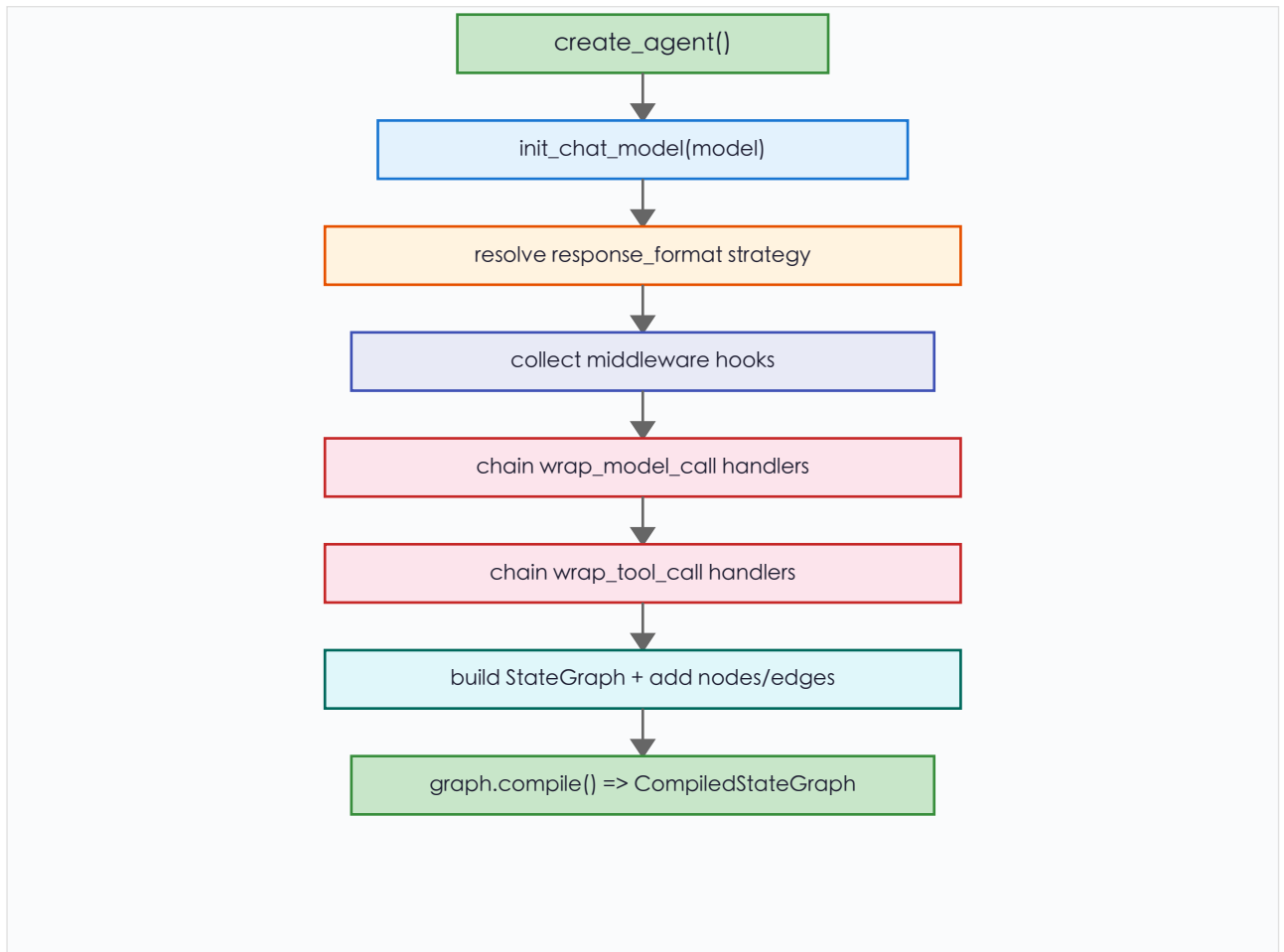
流程说明

- `START` -> `before_agent` : Agent 启动前执行一次（如初始化、资源加载）
- `before_model` : 每次模型调用前执行（如消息预处理、限流检查）
- `Model Node` : 调用 LLM，`wrap_model_call` 中间件可拦截此调用
- `after_model` : 模型返回后执行（如响应后处理、日志记录）
- `tool_calls` 判断 : 如果 `AIMessage` 包含 `tool_calls`，进入工具执行
- `Tools Node` : 并行执行工具，`wrap_tool_call` 中间件可拦截工具调用
- `Loop` : 工具执行完毕后回到 `before_model` 开始新一轮循环
- `after_agent` -> `END` : 循环结束后执行一次（如资源清理、最终处理）

四、核心流程图：create_agent 初始化

`create_agent()` 是构建的唯一入口函数。它接受模型、工具、中间件等参数，构建一个完整的并编译为可执行的 `CompiledStateGraph`。

Agent
StateGraph

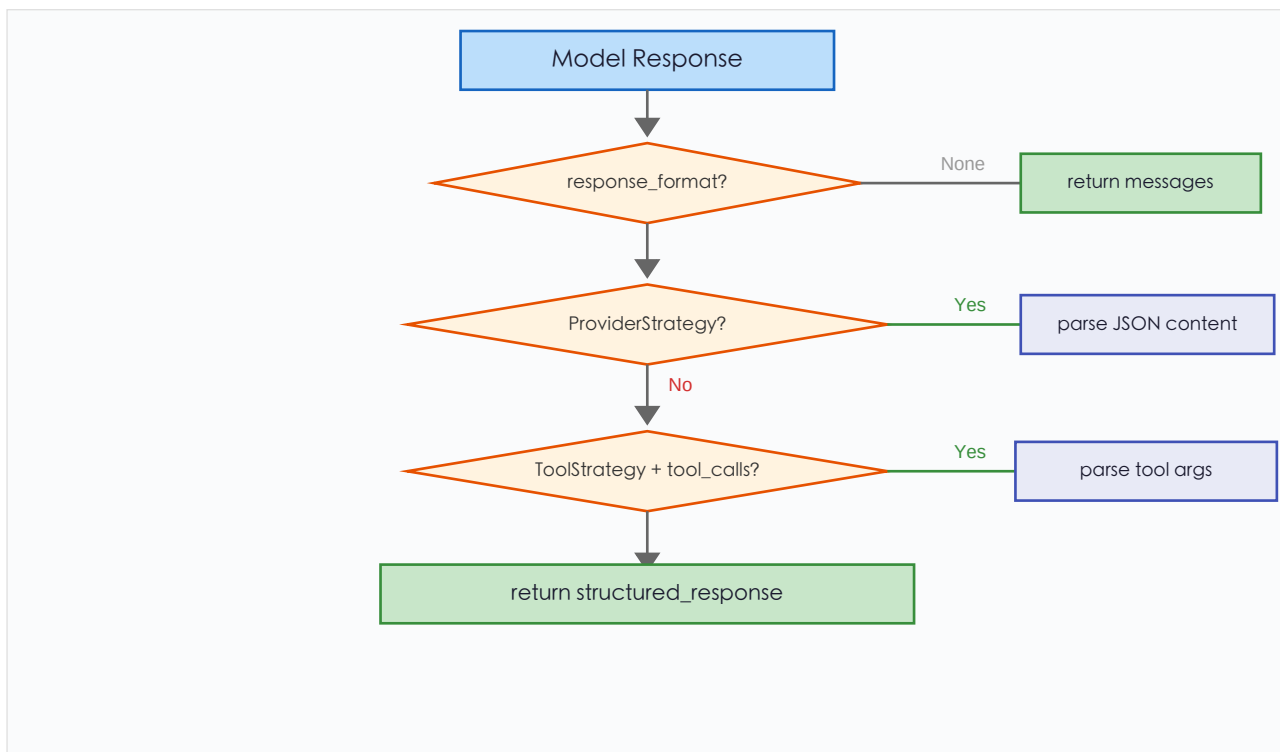


初始化步骤详解

- `init_chat_model`：将模型字符串（如 'openai:gpt-4o'）解析为 `BaseChatModel` 实例
- `resolve response_format`：确定结构化输出策略（`AutoStrategy/ToolStrategy/ProviderStrategy`）
- `collect middleware hooks`：遍历中间件，分类收集各生命周期钩子
- `chain handlers`：将同类钩子组合为嵌套的处理器链（洋葱模型）
- `build StateGraph`：创建状态图，添加节点（`model/tools/中间件`）和边（包括条件边）
- `graph.compile()`：编译状态图，生成可执行的 `CompiledStateGraph`

五、核心流程图：结构化输出处理

langchain_v1 支持两种结构化输出策略：ProviderStrategy（利用提供商原生的 JSON Schema 能力）和 ToolStrategy（通过工具调用实现结构化输出）。AutoStrategy 会根据模型能力自动选择最佳策略。

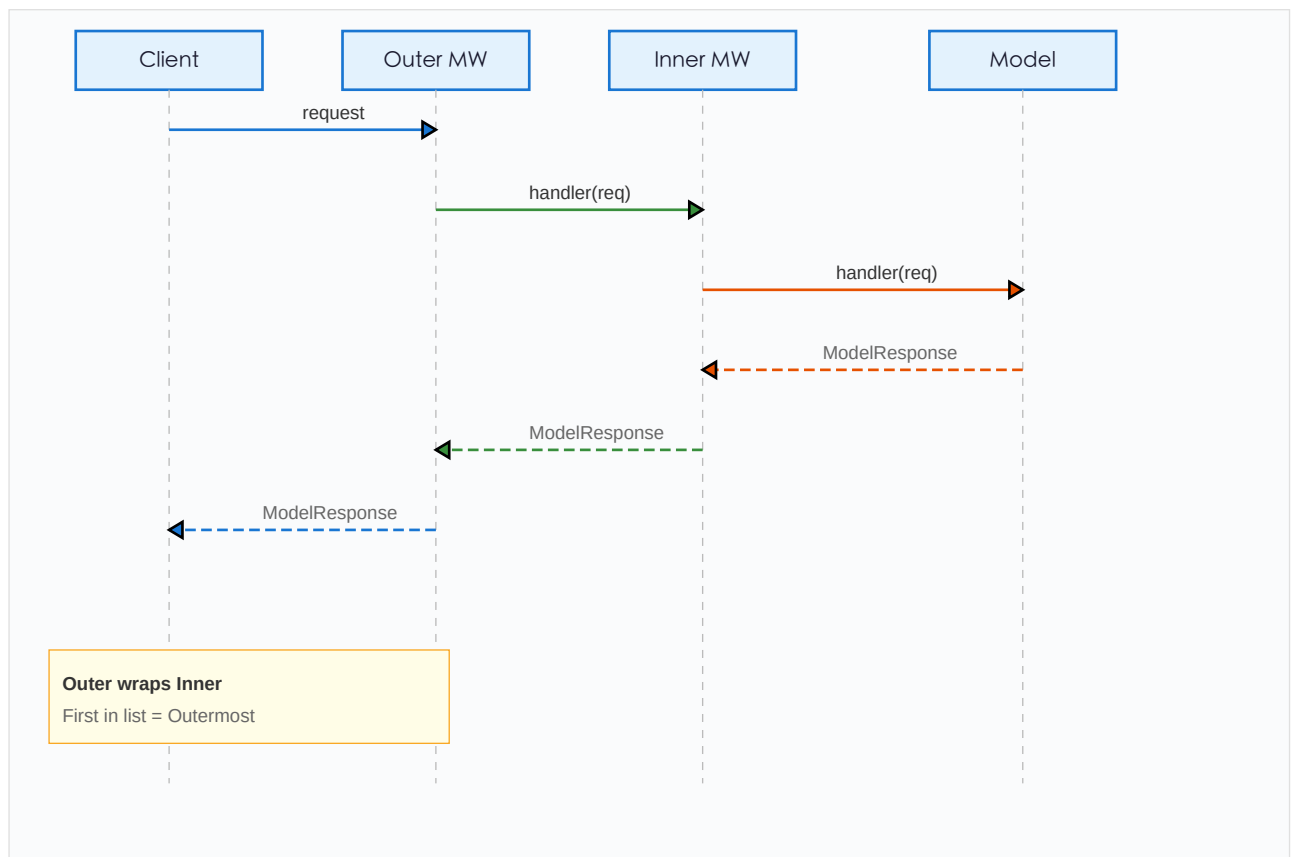


策略选择逻辑

- AutoStrategy：自动检测模型是否支持 provider 原生结构化输出。如支持则用 ProviderStrategy，否则回退到 ToolStrategy
- ProviderStrategy：利用 model.bind_tools() 的 response_format 参数，由提供商保证 JSON 格式合规
- ToolStrategy：将 schema 包装为虚拟工具（StructuredTool），强制模型调用该工具，从 tool_call args 中解析结构化响应

六、核心时序图：中间件组合模式

多个中间件的 `wrap_model_call` / `wrap_tool_call` 钩子采用洋葱模型组合，即列表中第一个中间件是最外层，最后一个是最内层。每个中间件通过 `handler` 回调链式调用下一层，最终到达实际的模型/工具执行。



组合细节

- 内部使用 `_chain_model_call_handlers()` 将多个 `handler` 函数通过 `compose_two()` 逐层嵌套
- 每层中间件可以：修改请求、多次调用 `handler`（重试）、跳过 `handler`（短路）、修改响应
- `Command` 从内层到外层累积，最终由 `_build_commands()` 统一处理
- 同步和异步版本分别实现 (`_chain_model_call_handlers` / `_chain_async_model_call_handlers`)

七、核心类设计说明 — AgentMiddleware

AgentMiddleware 是所有中间件的基类，定义了 Agent 生命周期中可扩展的钩子方法。它使用三个泛型参数：StateT（状态类型）、ContextT（运行时上下文）、ResponseT（结构化响应类型）。

7.1 类签名

```
class AgentMiddleware(Generic[StateT, ContextT, ResponseT])
```

7.2 生命周期钩子

钩子方法	执行时机	执行频次	可返回类型
before_agent / abefore_agent	Agent 启动前	仅一次	dict None
before_model / abefore_model	每次模型调用前	每轮循环	dict None
wrap_model_call / awrap_model_call	拦截模型调用	每轮循环	ModelResponse AIMessage ExtendedModelResponse
after_model / aafter_model	模型返回后	每轮循环	dict None
wrap_tool_call / awrap_tool_call	拦截工具调用	每次工具调用	ToolMessage Command
after_agent / aafter_agent	Agent 结束后	仅一次	dict None

7.3 重要属性

- state_schema：中间件使用的自定义状态 TypedDict 类型，会与基础 AgentState 合并
- tools：中间件注册的额外工具列表
- name：中间件名称（默认为类名），用于状态图节点命名

7.4 装饰器快捷方式

除了继承 AgentMiddleware 基类外，还可以通过装饰器快速创建中间件：

装饰器	对应钩子	说明
@before_agent	before_agent	Agent 执行前的一次性逻辑
@before_model	before_model	模型调用前的预处理逻辑
@after_model	after_model	模型返回后的后处理逻辑
@after_agent	after_agent	Agent 结束后的清理逻辑

@wrap_model_call	wrap_model_call	拦截模型调用的中间件
@wrap_tool_call	wrap_tool_call	拦截工具调用的中间件
@dynamic_prompt	wrap_model_call	动态生成系统提示词
@hook_config	任意钩子	配置钩子行为（如 can_jump_to）

八、核心类设计说明 — create_agent 工厂函数

`create_agent()` 是 `langchain_v1` 最核心的 API，位于 `agents/factory.py`。它是一个纯函数（非类），接受声明式配置参数，返回编译后的 `StateGraph`。

8.1 函数签名

```
def create_agent(
    model: str | BaseChatModel,
    tools: Sequence[BaseTool | Callable | dict] | None = None,
    *,
    system_prompt: str | SystemMessage | None = None,
    middleware: Sequence[AgentMiddleware] = (),
    response_format: ResponseFormat | type | dict | None = None,
    state_schema: type[AgentState] | None = None,
    context_schema: type | None = None,
    checkpointer: Checkpointer | None = None,
    store: BaseStore | None = None,
    ...
) -> CompiledStateGraph
```

8.2 核心参数说明

参数	类型	说明
model	str BaseChatModel	模型标识，如 'openai:gpt-4o' 或 BaseChatModel 实例
tools	Sequence[...] None	工具列表，支持 BaseTool、Callable、dict（内置提供商工具）
system_prompt	str SystemMessage None	系统提示词，作为每次模型调用的前缀
middleware	Sequence[AgentMiddleware]	中间件序列，按顺序应用
response_format	ResponseFormat type None	结构化输出配置（Pydantic/TypedDict/dict）
checkpointer	Checkpointer None	状态持久化，用于多轮对话记忆
store	BaseStore None	跨线程数据持久化（跨会话/用户）

8.3 内部关键流程

- 使用 `_resolve_schema()` 合并所有中间件的 `state_schema` 和基础 `AgentState`
- 通过 `_chain_model_call_handlers()` 将 `wrap_model_call` 钩子组合为洋葱模型
- 构建 `StateGraph` 时根据中间件类型动态添加节点和条件边
- 使用 `_add_middleware_edge()` 支持中间件的 `jump_to` 条件跳转

九、核心类设计说明 — ModelRequest / ModelResponse

9.1 ModelRequest

ModelRequest 是模型调用的请求对象，封装了模型调用所需的全部信息。它是不可变设计，通过 override() 方法创建新的修改副本。

属性	类型	说明
model	BaseChatModel	要调用的聊天模型实例
messages	list[AnyMessage]	消息列表（不含系统消息）
system_message	SystemMessage None	系统消息
tools	list[BaseTool dict]	可用工具列表
response_format	ResponseFormat None	结构化输出格式
state	AgentState	当前 Agent 状态
runtime	Runtime[ContextT]	运行时上下文
model_settings	dict[str, Any]	额外模型设置
tool_choice	Any None	工具选择配置

不可变模式：直接属性赋值已被弃用，建议使用 request.override(model=new_model) 创建新实例。这种设计确保中间件链中各层看到的是独立的请求对象，避免副作用。

9.2 ModelResponse

ModelResponse 封装了模型调用的结果：

属性	类型	说明
result	list[BaseMessage]	模型返回的消息列表（通常包含一个 AIMessage）
structured_response	ResponseT None	解析后的结构化输出，如果指定了 response_format

9.3 ExtendedModelResponse

ExtendedModelResponse 是 wrap_model_call 中间件的扩展返回类型，除了包含 ModelResponse 外，还可以附带一个 Command 对象用于额外的状态更新。

十、核心类设计说明 — 结构化输出策略

langchain_v1 定义了三种结构化输出策略，通过 ResponseFormat 联合类型统一管理：

策略类	原理	适用场景
AutoStrategy	根据模型能力自动选择 Provider 或 Tool 策略	推荐默认使用，无需关心模型支持情况
ProviderStrategy	利用模型提供商原生 JSON Schema（如 OpenAI response_format）	模型原生支持结构化输出时性能最优
ToolStrategy	将 schema 包装为虚拟工具，通过 tool_call 解析	通用兼容方案，支持所有模型

辅助类

- _SchemaSpec：Schema 描述封装，支持 Pydantic、dataclass、TypedDict、JSON Schema 四种类型
- OutputToolBinding：Tool 策略下的工具绑定信息，包含 schema、tool 实例和解析逻辑
- ProviderStrategyBinding：Provider 策略下的绑定信息，从 AIMessage 内容解析 JSON
- StructuredOutputError：结构化输出异常体系，包含 ValidationError 和 MultipleOutputsError

十一、核心类设计说明 — init_chat_model 工厂

init_chat_model() 是统一的聊天模型初始化入口，支持通过字符串标识符（如 'openai:gpt-4o'）创建任意提供商的模型实例。

11.1 支持的提供商 (25+)

提供商 Key	集成包	模型示例
openai	langchain-openai	GPT-4o, o3-mini 等
anthropic	langchain-anthropic	Claude Sonnet, Opus 等
google_vertexai	langchain-google-vertexai	Gemini 系列
google_genai	langchain-google-genai	Google GenAI
bedrock / bedrock_converse	langchain-aws	Amazon Bedrock 系列
azure_openai	langchain-openai	Azure OpenAI
ollama	langchain-ollama	本地模型 (Llama 等)
deepseek	langchain-deepseek	DeepSeek 系列
groq	langchain-groq	Groq 推理加速
mistralai	langchain-mistralai	Mistral 系列
xai	langchain-xai	Grok 系列
cohere	langchain-cohere	Command 系列
fireworks	langchain-fireworks	Fireworks AI
together	langchain-together	Together AI
perplexity	langchain-perplexity	Sonar 系列

11.2 _ConfigurableModel

当不指定固定模型或设置 configurable_fields 时，init_chat_model 返回 _ConfigurableModel。这是一个 Runnable 的延迟代理，在实际调用时才根据 config 参数实例化底层模型。支持队列化的声明式操作（如 bind_tools）在模型实例化后批量应用。

十二、核心类设计说明 — init_embeddings 工厂

init_embeddings() 提供统一的嵌入模型初始化接口，设计模式与 init_chat_model 一致。

提供商 Key	集成包	模型示例
openai	langchain-openai	text-embedding-3-small 等
azure_openai	langchain-openai	Azure OpenAI Embeddings
bedrock	langchain-aws	Amazon Titan Embed
cohere	langchain-cohere	embed-english-v3.0 等
google_vertexai	langchain-google-vertexai	VertexAI Embeddings
google_genai	langchain-google-genai	Google GenAI Embeddings
huggingface	langchain-huggingface	HuggingFace Embeddings
mistralai	langchain-mistralai	Mistral Embeddings
ollama	langchain-ollama	本地嵌入模型

十三、内置中间件一览

langchain_v1 提供了 17 种预构建中间件，覆盖了 Agent 开发中常见的横切关注点：

中间件	钩子类型	功能描述
ModelRetryMiddleware	wrap_model_call	模型调用失败时自动重试，支持指数退避和异常过滤
ModelFallbackMiddleware	wrap_model_call	模型调用失败时自动切换到备选模型
ModelCallLimitMiddleware	before_model	限制模型调用次数（按线程/按运行），防止无限循环
ToolRetryMiddleware	wrap_tool_call	工具调用失败时自动重试，支持指数退避
ToolCallLimitMiddleware	after_model	限制工具调用次数（按线程/按运行/按工具名）
HumanInTheLoopMiddleware	after_model	人工审核工具调用，支持审批/编辑/拒绝
SummarizationMiddleware	before_model	自动摘要过长的对话历史，防止上下文溢出
PIIMiddleware	before_model + after_model	PII 检测与处理（邮箱/信用卡/IP/URL 等）
ShellToolMiddleware	wrap_tool_call + tools	提供持久化 Shell 会话工具，支持 Docker/主机/Codex 沙箱
ContextEditingMiddleware	before_model	工具使用后编辑上下文（如清除旧工具结果）
TodoListMiddleware	tools	提供 TODO 列表管理工具
FilesystemFileSearchMiddleware	tools	提供文件系统搜索工具
LLMToolEmulator	wrap_model_call	用 LLM 模拟工具行为（测试/模拟场景）
LLMToolSelectorMiddleware	wrap_model_call	用 LLM 智能选择最相关的工具子集

中间件分类

- 可靠性类：ModelRetryMiddleware, ModelFallbackMiddleware, ToolRetryMiddleware
- 限流控制类：ModelCallLimitMiddleware, ToolCallLimitMiddleware
- 安全合规类：PIIMiddleware, HumanInTheLoopMiddleware
- 上下文管理类：SummarizationMiddleware, ContextEditingMiddleware
- 工具扩展类：ShellToolMiddleware, TodoListMiddleware, FilesystemFileSearchMiddleware
- 增强型类：LLMToolEmulator, LLMToolSelectorMiddleware

十四、设计哲学与总结

14.1 架构优势

- 极致精简：整个框架仅约 30 个源文件，核心 API 只有 create_agent() 一个函数
- 中间件即功能：所有横切关注点（重试/限流/PII/HITL 等）都通过可插拔中间件实现
- 声明式配置：create_agent() 采用纯声明式参数，内部自动构建 StateGraph
- 类型安全：大量使用 Generic、TypedDict、Protocol 和 Annotated，提供强类型保障
- 洋葱模型组合：wrap_model_call/wrap_tool_call 采用经典的中间件洋葱模型

14.2 与 langchain-classic 的对比

特性	langchain-classic	langchain_v1
核心概念	Chain（链式调用）	Agent + Middleware（中间件模式）
模块数量	chains/agents/memory/retrievers 等数十个模块	仅 6 个精简模块
编排引擎	内置简单链式 + Agent 循环	基于 LangGraph StateGraph
扩展方式	继承 Chain/Agent 子类	实现 AgentMiddleware 钩子
状态管理	ConversationMemory 等	StateGraph + Checkpointer
结构化输出	output_parsers	ResponseFormat 策略模式
类型安全	较弱	强类型（泛型+TypedDict+Protocol）

14.3 核心依赖关系

langchain_v1 的依赖层次如下：

- langchain-core：提供 BaseChatModel、BaseTool、BaseMessage 等核心抽象
- langgraph：提供 StateGraph、Runtime、Checkpointer、ToolNode 等编排能力
- langsmith：提供 @traceable 装饰器用于可观测性
- pydantic：提供数据验证和 JSON Schema 生成